

M4 — What this block gets you

- ✓ ELT reframed—land raw in BigQuery, transform with SQL inside the warehouse, not in an SSIS data flow
- ✓ Scripting, UDFs, stored procedures— your T-SQL instincts transfer; watch dialect, not concepts
- ✓ Scheduled queries— the SQL Agent instinct in one click; the closing demo makes it concrete
- ✓ Dataform— your architecture doc's Silver→Gold tool: sqlx models, the ref() graph, assertions, releases
- ✓ Hands-on: watch a real repo end to end, then build your own SQL workflow in the anchor lab

This afternoon, block by block

14:45–15:10 M4 — ELT in BigQuery and the Dataform intro; fast through scripting, UDFs, sprocs

15:10–15:25 Jeff's dataform-demo repo, live — a real Silver→Gold pipeline in git

DEMO

15:25–16:05 Lab: Create and Execute a SQL Workflow in Dataform — the anchor lab, full slot

LAB

16:05–16:20 M5 + M6 compressed — ETL options and the automation ladder

16:20–16:30 Close: schedule the derived-table query — one click, your nightly SQL Agent job

DEMO

Once data is loaded into BigQuery, there are multiple ways to transform it



Procedural language



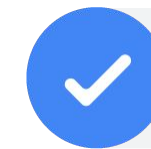
Scheduled query



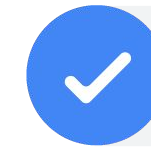
Jupyter Notebook



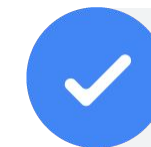
Dataform



Scripting and programming



Scheduling capabilities



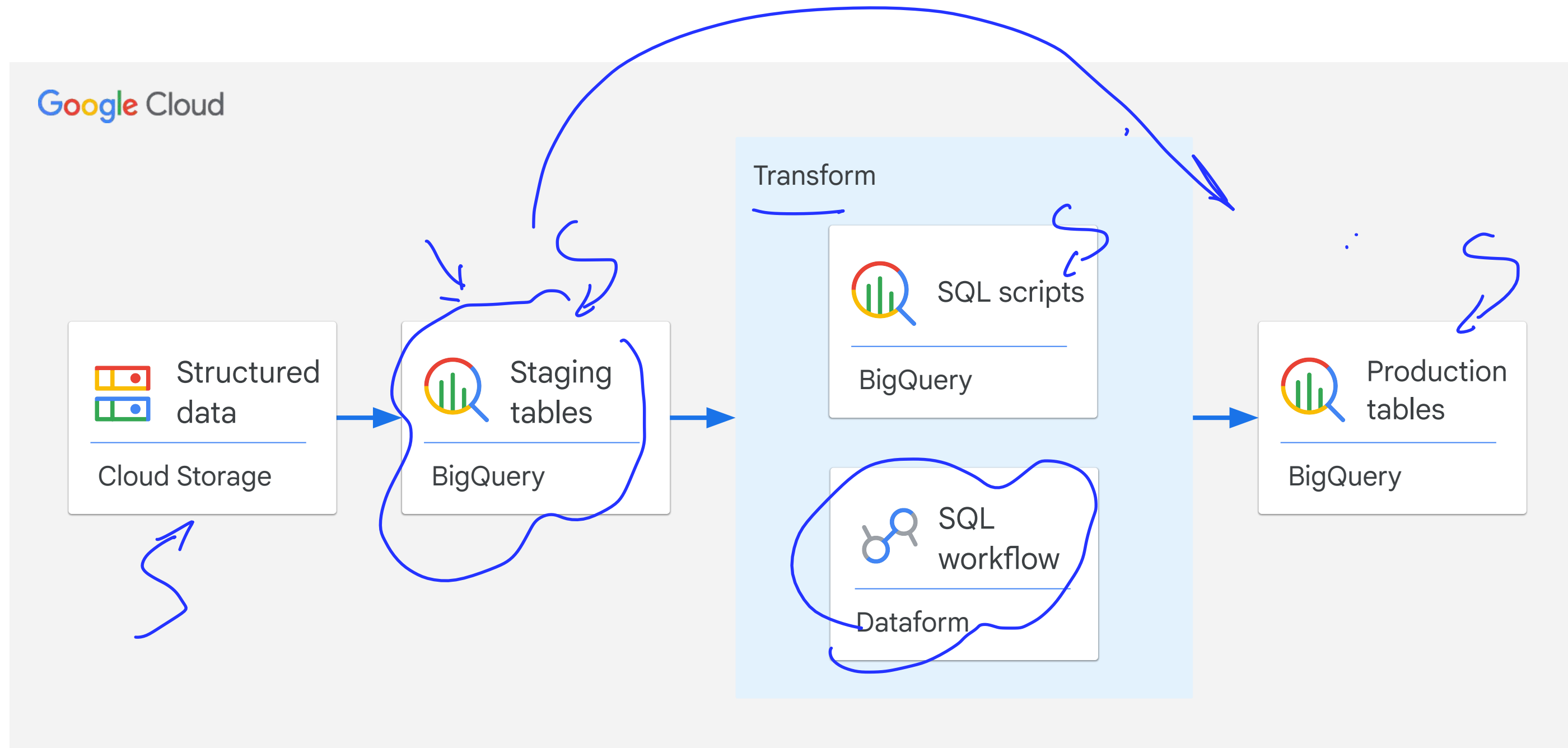
Dependency management

Extract and load

Extract, load,
transform

Extract,
transform, load

Use an ELT pipeline to apply transformations after data is staged in BigQuery



BigQuery supports procedural language queries to perform multiple statements in a sequence

```
-- create variables
DECLARE book_name STRING DEFAULT 'Ulysses';
DECLARE book_year INT64 DEFAULT 1922;

-- Create a temporary table called Books.
EXECUTE IMMEDIATE
  "CREATE TEMP TABLE Books
    (title STRING, publish_date INT64)";

-- add a row for Ulysses, using the variables
-- declared and the ? placeholder
EXECUTE IMMEDIATE
  "INSERT INTO Books (title, publish_date) VALUES(?, ?)"
  USING book_name, book_year;
```

Procedural language in BigQuery:

- Runs multiple statements in a sequence with shared state.
- Automates management tasks, such as creating or dropping tables.
- Implements complex logic using programming constructs like `IF` and `WHILE`.
- Can declare user-created variables or reference BigQuery's system variables.
- Can use transactions (`COMMIT`, `ROLLBACK`).

User-defined functions (UDFs) transform data by using SQL or JavaScript

```
# create a permanent user-defined function
CREATE FUNCTION dataset_name.AddFourAndDivide
(x INT64, y INT64) RETURNS FLOAT64 AS (
  (x + 4) / y
);
```

```
# use the function in your SQL script
SELECT
  val, dataset_name.AddFourAndDivide(val, 2)
FROM ...;
```

Usage notes:

- Can be persistent (CREATE FUNCTION) or temporary (CREATE TEMPORARY FUNCTION).
- Use SQL where possible. Use JavaScript if the same cannot be accomplished with SQL.
- JavaScript UDFs can use additional input libraries.
- The `bigquery-utils` project and the `bigquery-public-data.persistent_udfs` public dataset contain community-contributed UDFs.

Stored procedures execute a collection of SQL statements

create a stored procedure

CREATE OR REPLACE PROCEDURE

dataset_name.create_customer()

BEGIN

DECLARE id STRING;

SET id = GENERATE_UUID();

INSERT INTO dataset_name.customers (customer_id)

VALUES(id);

SELECT FORMAT("Created customer %s", id);

END

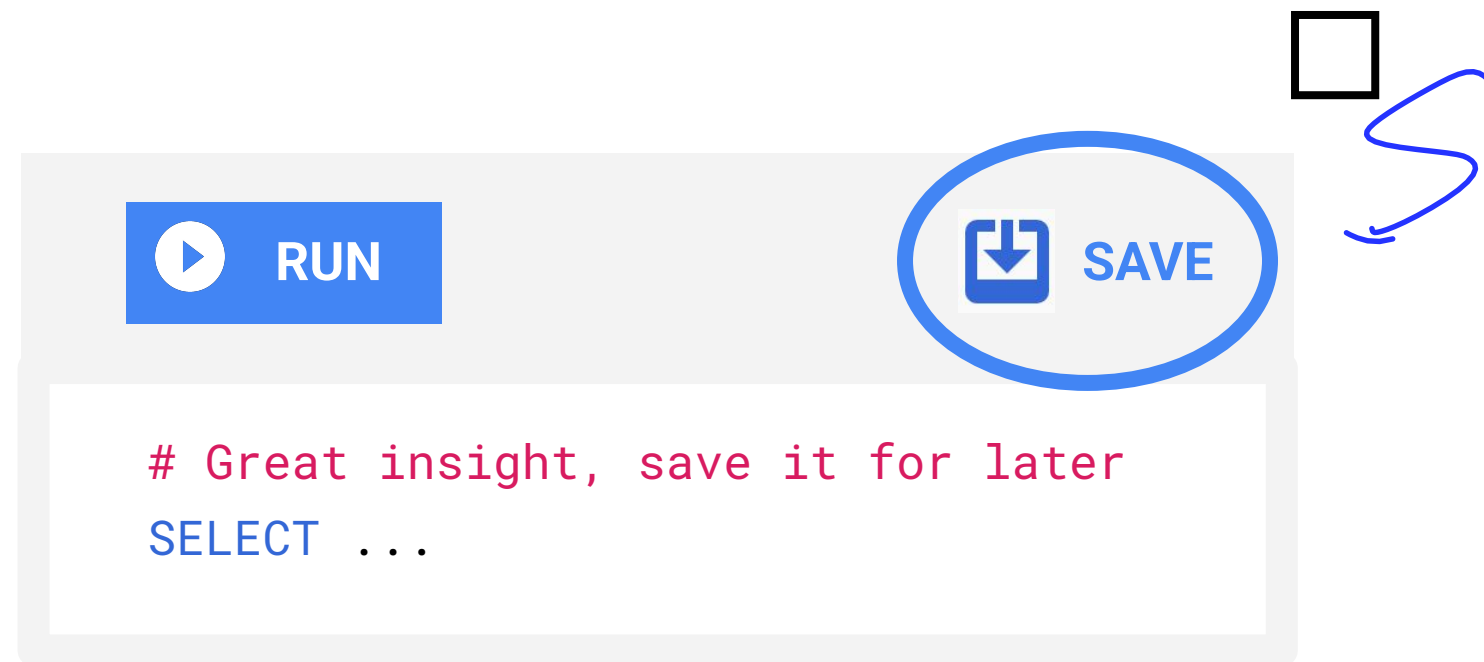
call the procedure in your SQL script

CALL dataset_name.create_customer();

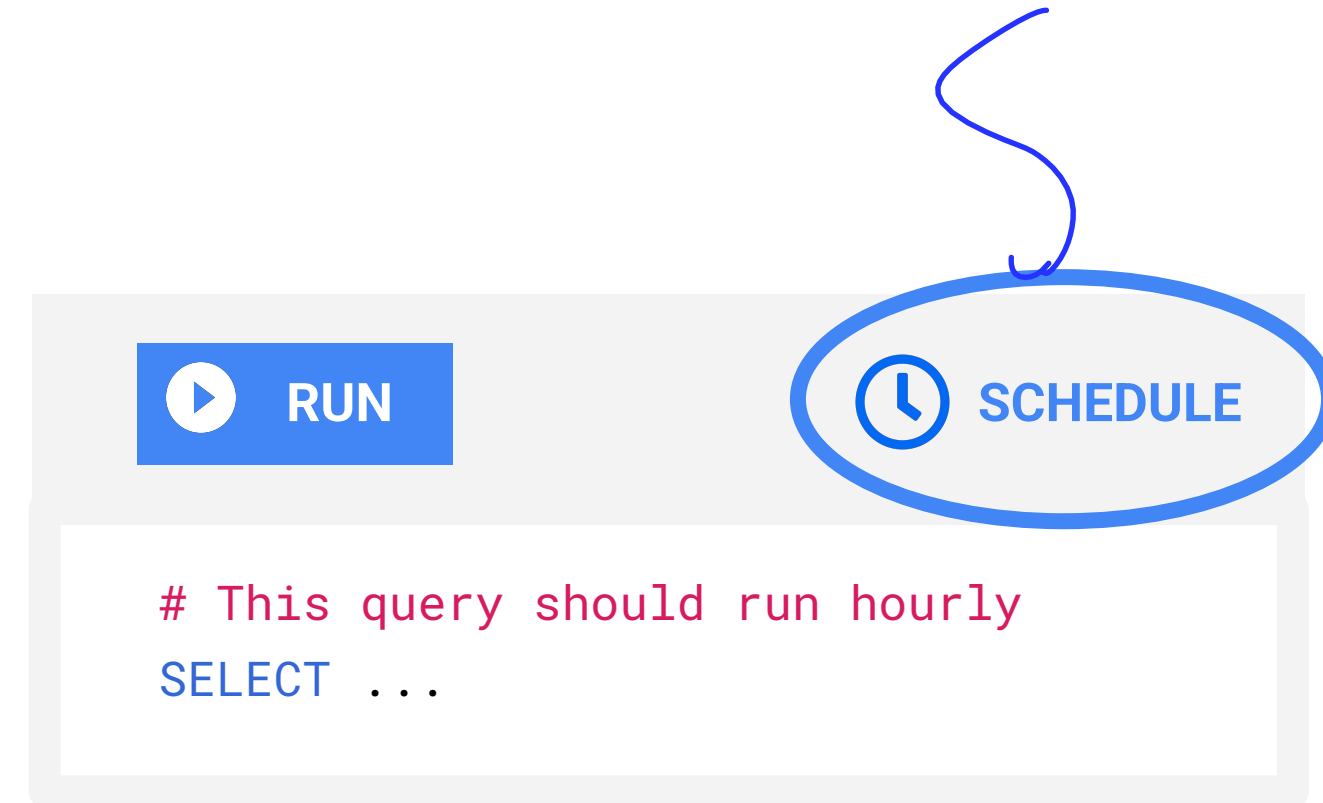
Usage notes:

- Can be called from queries or other stored procedures.
- Accept input values and return values as output.
- Can access or modify data across multiple datasets.
- Can contain multi-statement queries and support procedural language statements.

Save your queries and schedule them for repeating use cases

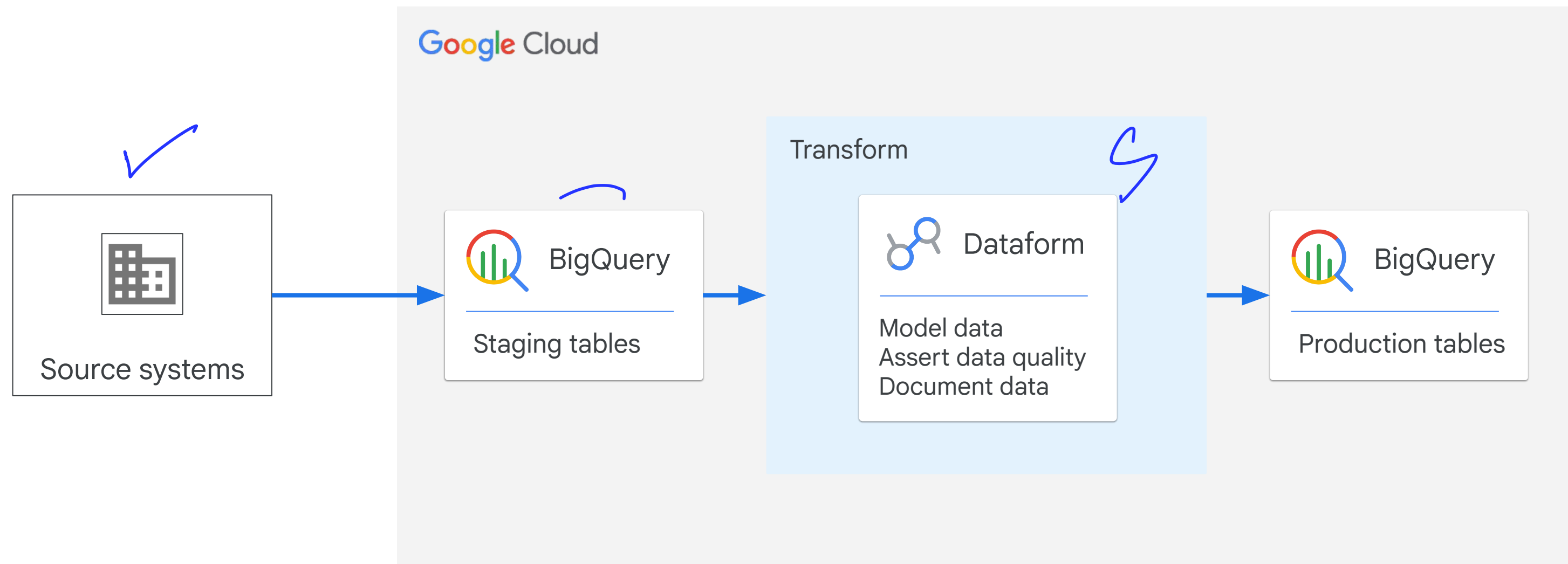


- Is visible in Explorer > Queries.
- Supports version control.
- Share with users or groups.
- Download as `.sql` file.
- Upload other queries from `.sql` files.



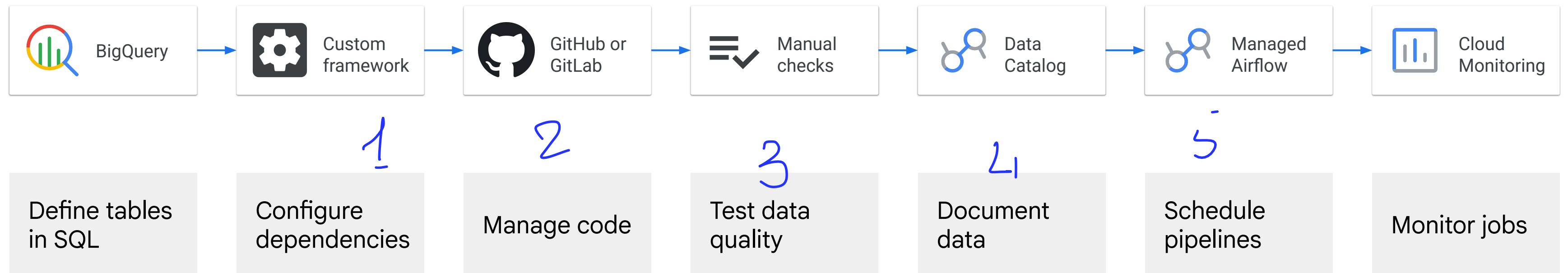
- Set repeat frequency.
- Set start and end times.
- Specify destination for query results.
- Can use runtime parameters.
- **Tip:** use Dataform for SQL workflows.

Dataform is a serverless framework to develop and operationalize ELT pipelines in SQL

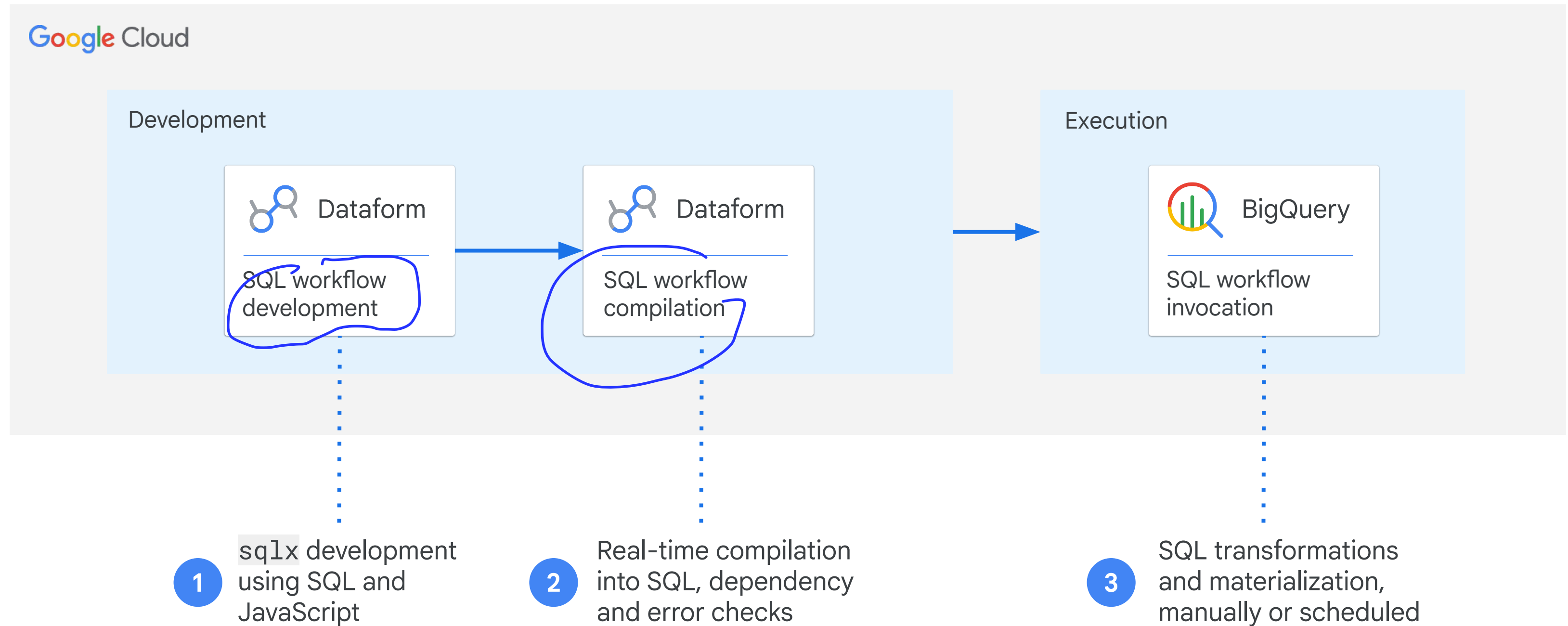


Dataform unifies data transformation, assertion, and automation in BigQuery

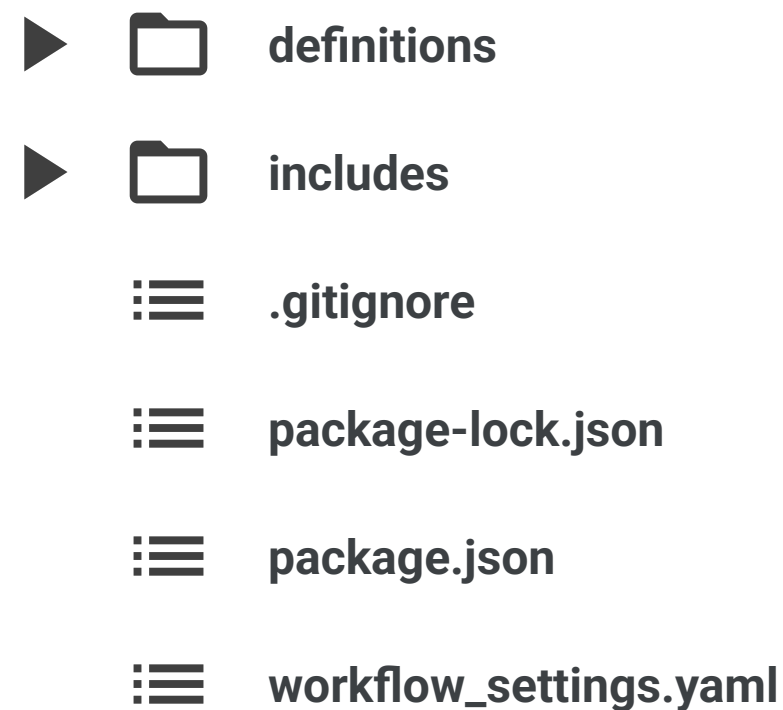
Without Dataform, this would be a time-consuming and error-prone process



SQL workflow development and compilation run in Dataform, workflow execution runs in BigQuery



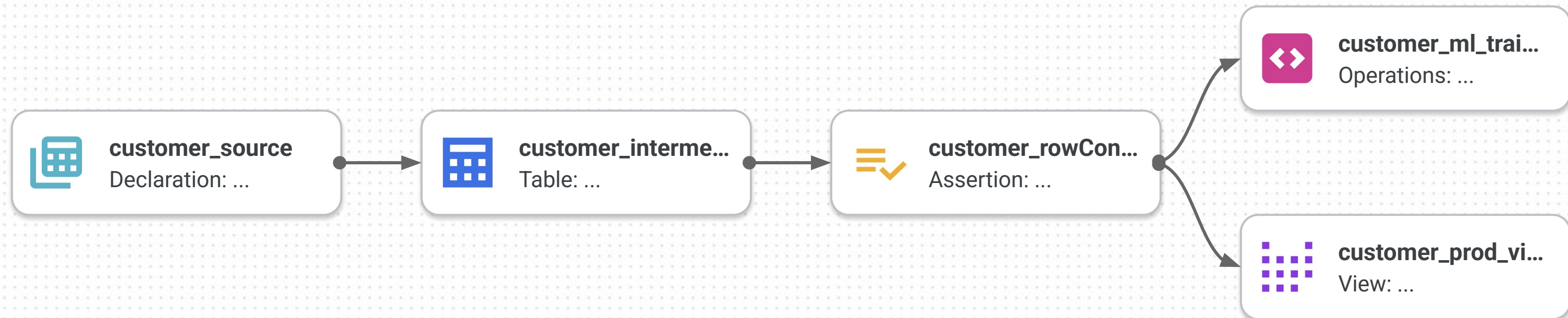
Development happens in workspaces with default files and folders



You can add custom files like `README.md` as well.

- **definitions** folder:
contains `sqlx` files.
- **includes** folder:
contains JavaScript files.
- **.gitignore** file:
lists files and directories to ignore during Git commits.
- **package.json** file (optional):
defines JavaScript packages and required versions.
- **package-lock.json** file (optional):
contains installed packages with exact versions.
- **workflow_settings.yaml** file:
contains basic settings required to compile the project.

The compiled graph visualizes the SQL workflow with its definitions and dependencies



Demo: a real Dataform repo, end to end

15 minutes · github.com/jwdavis/dataform-demo, plus the Google Cloud console

1. Walk the repo tree: dataform.json, one constants file, definitions/ — declarations, staging, marts
2. Read the sqlx: a declaration (Bronze), staging cleanup (Silver), an aggregate mart (Gold)
3. Console: repository, GitHub connection, development workspace — your feature branch
4. Compiled graph: ref() calls become the dependency graph — your Silver→Gold pipeline as code
5. Add an assertion live, then release config + scheduled invocation — your CI/CD, translated

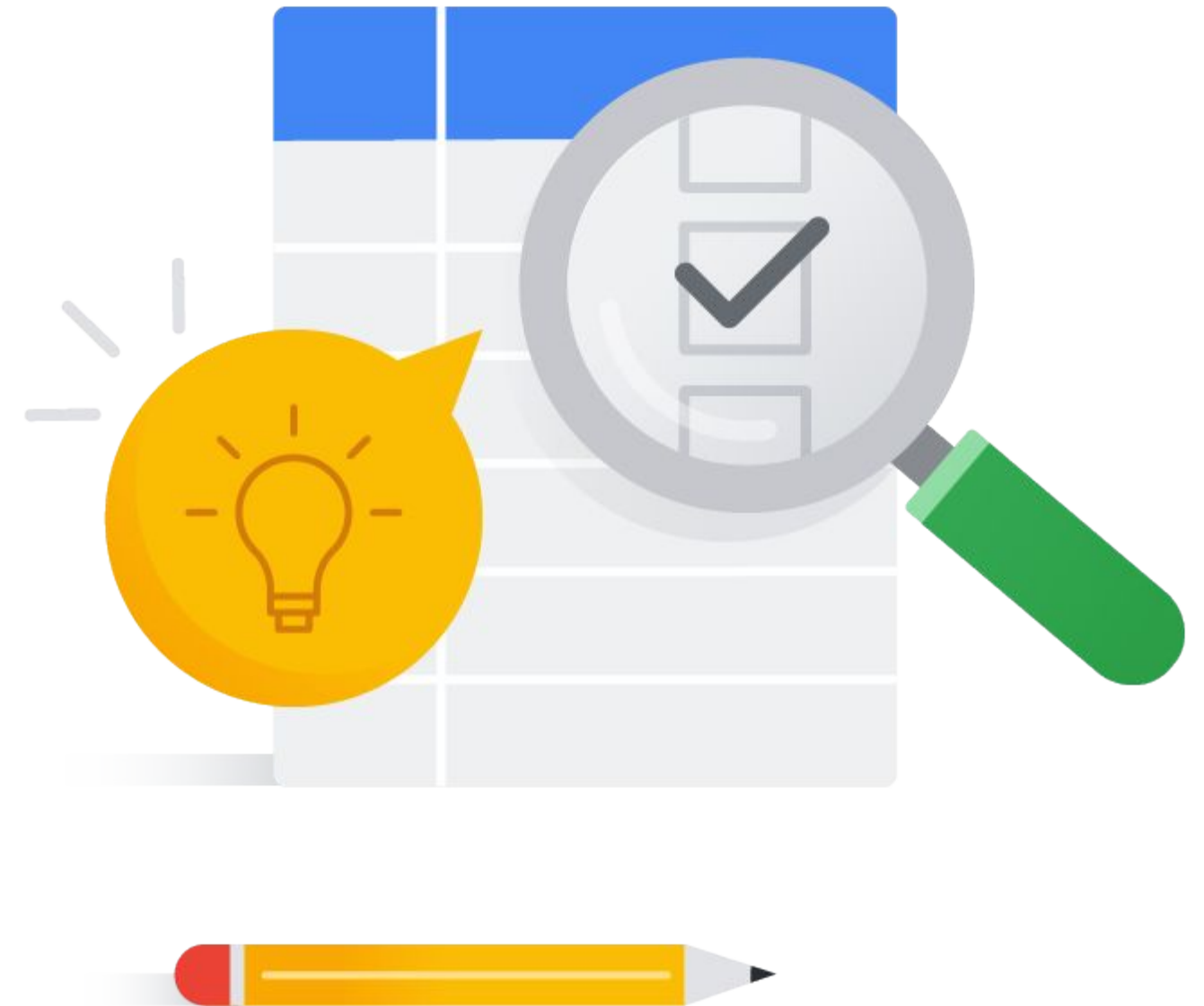
Why it matters — your architecture doc names Dataform for Silver→Gold — here is one running: raw → staging → marts, in git, deployable. The discipline you already run for SQL Server.

Lab: Create and Execute a SQL Workflow in Dataform

🕒 30 min

Learning objectives

- Create a Dataform repository.
- Create and initialize a Dataform development workspace.
- Create and execute a SQL workflow.
- View execution logs in Dataform.



Lab: Create and Execute a SQL Workflow in Dataform

40 minutes · Google Skills classroom — lab project, BigQuery Studio → Dataform

1. Create a Dataform repository — watch the region; a mismatch is the classic stall
2. Create and initialize a development workspace — files appear only after Initialize
3. Define a staging view and a mart table in sqlx, wired together with `ref()`
4. Execute the workflow and read the logs — actions run in dependency order, compiled SQL shown
5. Early finishers: add a `uniqueKey` or `rowConditions` assertion and watch it enter the graph

Why it matters — the one solid lab in the course — these 40 minutes are exactly what your migration does at work: SQL models in git, compiled into a graph, executed in order, tested in-pipeline.

Close: schedule the derived-table query

5 minutes · BigQuery Studio — Schedule → Create new scheduled query

1. Run the sales-by-zip query once into class.march_zip_sales
2. Schedule → Create new scheduled query: repeat daily at 12:01 AM
3. Destination: your class dataset; write preference Overwrite (WRITE_TRUNCATE)
4. Save, find it under Scheduled queries, trigger one manual run to prove it fires

Why it matters — a SQL statement, on a schedule, overwriting a table — your nightly SQL Agent job minus the server. Day 2 replaces hand-rolled derived tables with Dataform-managed marts.

M4 — What you now have

- ✓ ELT on BigQuery— land Bronze, let SQL build Silver and Gold inside the warehouse
- ✓ Dataform, hands-on— repo, workspace, sqlx models, the ref() graph, executed and logged
- ✓ The git discipline you already run— workspace = feature branch, release config = deployment
- ✓ A scheduled query overwriting a derived table nightly— SQL Agent translated, first automation
- ✓ Tomorrow: the medallion and governance— Gold marts feeding the Power BI you already have